FootTracker

API REST de statistiques de football

Projet Spring Boot – Rapport de projet
Mathis Rouy et [Nom du binôme]
Octobre 2026

Dépôt : https://github.com/Shadix95/FootTracker

1. Présentation du projet

FootTracker est une API REST développée en **Spring Boot 4** (Java 21). Elle récupère les données de l'API publique **football-data.org** (matchs, classements, buteurs), les enregistre dans une base de données, puis produit des **bilans statistiques** qui ne sont pas fournis directement par football-data.org.

Une interface web simple (HTML + Bootstrap) est fournie pour consulter les statistiques, et toute l'API est documentée avec **Swagger**.

Stack technique

Technologie	Utilisation
Spring Web MVC	Controllers REST
RestClient	Appels HTTP vers football-data.org
Spring Data JPA / Hibernate	Persistance des entités en base
H2 / PostgreSQL	H2 par défaut (fichier local), PostgreSQL via un profil Spring et Docker
Spring Cache + Caffeine	Cache de 5 minutes sur les appels externes
@Scheduled	Synchronisation automatique toutes les 30 minutes
Lombok	Réduction du code répétitif (getters, setters, constructeurs)
springdoc-openapi	Documentation Swagger générée automatiquement
JUnit 5, Mockito, MockMvc	22 tests automatisés
Bootstrap 5	Interface web minimaliste

2. Lancer le projet (pour la correction)

Prérequis

- Java 21 (le wrapper Maven `mvnw` est fourni, pas besoin d'installer Maven).
- Une clé gratuite football-data.org : https://www.football-data.org/client/register
- Docker, uniquement pour l'option PostgreSQL.

Option A : base H2 (le plus simple, aucune installation)

1. Définir la variable d'environnement `FOOTBALL_DATA_TOKEN` avec la clé (dans IntelliJ : Run > Edit Configurations > Environment variables).
2. Ou lancer l'application en ligne de commande (PowerShell) :

```
$env:FOOTBALL_DATA_TOKEN="votre_cle"
.\mvnw spring-boot:run
```

3. Ouvrir http://localhost:8080 : la base se remplit automatiquement 10 secondes après le démarrage (environ 25 secondes pour les deux championnats).

Option B : base PostgreSQL

1. Démarrer PostgreSQL et Adminer avec Docker :

```
docker compose up -d
```

2. Lancer l'application avec le profil postgres (dans IntelliJ : champ Active profiles = postgres) :

```
.\mvnw spring-boot:run "-Dspring-boot.run.profiles=postgres"
```

Initialisation de la base de données

Aucun script SQL n'est à exécuter :

- **Les tables** sont créées automatiquement par Hibernate au démarrage (`spring.jpa.hibernate.ddl-auto=update`).
- **Les données** sont importées par la synchronisation automatique, lancée 10 secondes après le démarrage puis toutes les 30 minutes. Elle peut aussi être déclenchée à la main avec le bouton « Synchroniser » de l'interface ou `POST /api/sync`.
- Les championnats synchronisés sont configurés dans `application.properties` : `football-data.competitions=PL,FL1`.

URLs utiles

URL	Contenu
http://localhost:8080	Interface web
http://localhost:8080/swagger-ui.html	Documentation Swagger (tester les endpoints)
http://localhost:8080/h2-console	Console H2 (JDBC URL : `jdbc:h2:file:./data/foottracker`, user `sa`, mot de passe vide)
http://localhost:8081	Adminer pour PostgreSQL (système PostgreSQL, serveur `postgres`, user, mot de passe et base : `foottracker`)

Lancer les tests : `./mvnw test` (ils utilisent une base H2 en mémoire et ne nécessitent ni clé API ni Docker).

3. Architecture

Le projet suit une architecture en couches classique : le controller reçoit la requête, le service fait les calculs, le repository accède à la base.

```
football-data.org → FootballDataClient → SyncService → Base (H2 / PostgreSQL)
Base → Repositories → Services de statistiques → Controllers → JSON / Front
```

Package	Rôle
client	FootballDataClient : appels à football-data.org, cache et limitation du débit
config	Configuration du RestClient (URL, clé API) et de Swagger
dto.api	Records qui reçoivent le JSON de football-data.org
dto.stats	Records renvoyés par notre API (statistiques calculées)
entity	Entités JPA : Team, Match, Scorer, FavoriteTeam
repository	Interfaces Spring Data JPA (méthodes dérivées et requêtes JPQL)
service	Synchronisation et calcul des statistiques
controller	Endpoints REST

Package	Rôle
exception	GlobalExceptionHandler : réponses d'erreur JSON uniformes

Les DTO (forme imposée par l'API externe) sont séparés des entités (forme choisie pour notre base) : un changement côté football-data.org n'impacte que la couche DTO et la synchronisation.

4. Les endpoints de l'API

Tous les endpoints sont aussi documentés et testables dans Swagger. `{code}` est le code d'une compétition (FL1 = Ligue 1, PL = Premier League).

Statistiques calculées (valeur ajoutée)

Méthode	URL	Description
GET	/api/competitions/{code}/standings	Classement calculé à partir des matchs. Paramètre optionnel `?matchday=N` pour le classement après la journée N
GET	/api/competitions/{code}/teams/{teamId}/stats	Bilan d'une équipe : moyennes de buts, clean sheets, domicile/extérieur, forme
GET	/api/competitions/{code}/teams/{teamId}/ranking-evolution	Position et points de l'équipe après chaque journée
GET	/api/competitions/{code}/scorers/stats	Buteurs : buts par match, buts hors penalty, part des buts de l'équipe
GET	/api/competitions/{code}/head-to-head?team1=X&team2=Y	Face-à-face entre deux équipes
GET	/api/competitions/{code}/matchdays/{matchday}/summary	Bilan automatique d'une journée

Équipes favorites

Méthode	URL	Description
GET	/api/favorites	Liste des favoris avec forme, dernier et prochain match
POST	/api/favorites/{teamId}	Ajoute une équipe aux favoris (201 Created)
DELETE	/api/favorites/{teamId}	Retire une équipe des favoris (204 No Content)

Synchronisation

Méthode	URL	Description
POST	/api/sync	Synchronise toutes les compétitions configurées

Méthode	URL	Description
POST	/api/sync/{code}	Synchronise une compétition

Données brutes (proxy football-data.org, cache 5 min)

Méthode	URL	Description
GET	/api/raw/competitions/{code}/standings	Classement officiel
GET	/api/raw/competitions/{code}/matches	Matchs de la saison
GET	/api/raw/competitions/{code}/scorers	Meilleurs buteurs

5. Notre valeur ajoutée

football-data.org fournit des données brutes : la liste des matchs avec leurs scores, un classement et une liste de buteurs. **Toutes les statistiques ci-dessous sont calculées par notre application** à partir des matchs stockés en base.

Bilan des équipes

- **Buts marqués et encaissés par match** : total des buts divisé par le nombre de matchs joués.
- **Clean sheets** : nombre de matchs sans but encaissé.
- **Bilan domicile / extérieur** : victoires, nuls, défaites, buts et points séparés selon le lieu du match.
- **Forme sur les 5 derniers matchs** : résultats (V/N/D) du plus récent au plus ancien.
- **Évolution du classement par journée** : pour chaque journée N, le classement est recalculé avec uniquement les matchs des journées 1 à N, et on relève la position de l'équipe.

Le classement est trié par points, puis différence de buts, puis buts marqués. Les règlements officiels utilisent parfois d'autres critères (confrontations directes), ce qui peut créer de rares écarts avec le classement officiel.

Bilan des buteurs

- **Buts par match** : buts divisés par matchs joués.
- **Buts hors penalty** : buts moins penaltys marqués.
- **Part des buts de l'équipe** : buts du joueur divisés par le total des buts de son équipe, calculé à partir des matchs en base (exemple : 21 buts sur 70 = 30 %).

Bonus

- **Face-à-face** : victoires de chaque équipe, nuls, buts marqués et liste des confrontations.
- **Bilan automatique de journée** : nombre de buts, moyenne par match, répartition domicile / nul / extérieur, plus large victoire, match le plus prolifique, matchs restant à jouer.
- **Équipes favorites** : seule donnée propre à notre application (table favorite_team). Pour chaque favori : forme, dernier match joué et prochain match, toutes compétitions confondues.

Respect du quota de l'API

L'offre gratuite limite à 10 requêtes par minute. Trois mécanismes le garantissent :

- **La base de données** : les statistiques lisent la base, jamais l'API directement.
- **La synchronisation planifiée** (`@Scheduled`) : l'API n'est interrogée que toutes les 30 minutes.

- **Le cache Caffeine** (`@Cacheable`, 5 min) et un **délai minimal de 6 secondes** entre deux appels dans le client.

6. Gestion des erreurs

Un `@RestControllerAdvice` intercepte les exceptions et renvoie toujours le même format JSON (status, error, message, path, timestamp).

Code	Cas
400	Paramètre invalide ou manquant (ex. `/teams/abc/stats`), même équipe dans un face-à-face
403	Clé API invalide ou compétition non incluse dans l'offre gratuite
404	Équipe, journée ou URL inconnue, code de compétition erroné
405	Mauvaise méthode HTTP (ex. GET sur /api/sync)
429	Quota de 10 requêtes par minute dépassé
502 / 503	Erreur inattendue de football-data.org / API injoignable
500	Erreur interne (détail dans les logs, pas exposé au client)

7. Tests

22 tests automatisés (`./mvnw test`), exécutés sur une base H2 en mémoire :

- Lecture du JSON de football-data.org dans les records (sans appel réseau).
- Synchronisation avec un client mocké (Mockito) : vérification de l'absence de doublons.
- Calculs statistiques sur un mini-championnat dont les résultats ont été vérifiés à la main.
- Bonus : face-à-face, bilan de journée, favoris.
- Gestion des erreurs avec MockMvc (403, 429, 404, 400, 405) et disponibilité de la documentation Swagger.

8. Collaboration avec Git

- Dépôt GitHub partagé : https://github.com/Shadix95/FootTracker
- Une branche par fonctionnalité (`feat/add-restclient-api`, `feat/add-calcul-stats`…) en plus des branches personnelles.
- Intégration dans `main` par **Pull Requests** (#1, #2, #3).
- Messages de commit au format Conventional Commits (`feat: …`), un commit par étape.
- Les secrets ne sont jamais versionnés : la clé API est une variable d'environnement, et la base locale (`data/`) est dans le `.gitignore`.

9. Compétences acquises

- **Spring Boot** : injection de dépendances par constructeur, beans, configuration externalisée et profils.
- **Consommer une API REST** avec RestClient et transformer le JSON en records Java.
- **Distinguer DTO et entités**, et convertir l'un vers l'autre lors de la synchronisation (upsert).

- **JPA / Hibernate** : entités, relations @ManyToOne / @OneToOne, méthodes dérivées et requêtes JPQL.
- **Tâches planifiées et cache** pour respecter les contraintes d'une API externe.
- **Exposer des points d'entrée REST adaptés** : verbes HTTP (GET, POST, DELETE), codes de retour (200, 201, 204), paramètres de chemin et de requête.
- **Gestion centralisée des erreurs** avec @RestControllerAdvice.
- **Documentation** d'une API avec OpenAPI / Swagger.
- **Tests** : tests d'intégration Spring, mocks avec Mockito, tests HTTP avec MockMvc.
- **Changer de base de données** sans modifier le code Java grâce à JPA, et lancer PostgreSQL avec Docker Compose.
- **Travailler en binôme avec Git** : branches, Pull Requests et intégration dans main.

10. Limites et améliorations possibles

- Pas d'authentification : les favoris sont communs à toute l'application.
- Seule la saison en cours est synchronisée : les face-à-face ne couvrent pas les saisons précédentes.
- Départage du classement simplifié (points, différence de buts, buts marqués).
- Migrations de schéma gérées par Hibernate ; un outil comme Flyway serait préférable en production.